

The heart of updatelessness about computations

Martín Soto

September 4, 2023

With updatelessness we want to be fair to other worlds. But other logical worlds don't seem "as real" as other empirical worlds. Empirical worlds can be understood "on their own terms", thanks to logical omniscience. They are "a real possibility". On the contrary, logical worlds cannot be understood "on their own terms", because we are using "real-world logic" (how computations actually turn out in our calculations) to understand and define them, and so this remnant of "non-agnostic / non-uncertain ground" remains.

For example, in LIs, although we are uncertain about derivability from PA, the traders we implement are still "computed using real-world computations" (that is, they are real Turing Machines, implementing real computations). Computations inside traders are the "certain ground" that remains. As a result, [real-world computations are still favored in LIs](#). This happens "because of the simplicity prior that chooses Traders", but of course any prior will be a simplicity prior eventually, so it actually happens more fundamentally because of using real computations.

Indeed, the difficulty of logical uncertainty can be understood as the difficulty of drawing a boundary between the "fixed ground, or form, or shape", and the "varying content". In the empirical case, all of logic is the form, and the outcomes of empirical coins are the content. In the logical case, we have to draw form / content distinctions inside logic, and we don't know how to do that. The LI framework chooses to set the boundary at "uncertainty about derivability from PA". Our modifications of Universal Inductors to have them be coherent in different ways give "a bit more terrain to form", artificially enforcing in the system certainty about some properties of real-world logic.

I'd like to peel the layers off this ground further and further, and see what's at its heart, or alternatively when everything breaks down. The ultimate goal is for logically possible worlds to "understand themselves from within, in their own terms". But this might be philosophically impossible from our real world.

Below I present my train of thought doing that.

Splitting logical inductors: When I observe a logical sentence, I want to spawn two copies of my LI. To one I will feed the real observation, and to the other one the opposite. I'll keep doing this and running them. When I want to condition on logical impossibilities, I will consult the LI who has actually received those observations (or the average of all such, or something more clever). The problem is, just as above, all of their traders are still implementing real computations. For example, in the LI where the first digit of pi is 2 instead of 3, and all other logical observations are the same, the trader defending the real digits of pi is still favored, because it's shorter to describe

that the one defending pi with the first digit changed.

Another way to put it is: flipping the syntactical truth value in an observation doesn't change that, when a Trader goes to compute the first digit of pi, it's still 3, and not 2. I want to somehow vary these internals in unison with the external observations.

Betting on finite computations: Instead of traders having opinions on derivability from PA (including about undecidable sentences), I can have them reason only about finite computations. For example, for any Turing Machine with description m , I introduce the syntactic variable $P_m(t)$, that represents the contents of the tape at time-step t when running m . This is a neat framework, but doesn't touch in any way the problem above. Changing observations, or more generally the framework we build around real computations, seems top-down, and not to peel layers off the ground. I need to get messier and look into the internals of computations (bottom-up).

The delicate workings of a Turing Machine: Consider the names of the basic operations of a Turing Machine: Left, Right, Write X, Read, etc. In our world, these names have some concrete actions associated with them. For example, Left corresponds to moving the tape head one square to the left. What if, in other worlds, these basic operations behave differently?

Imagine I tell you "computation A produces as output 1001, but computation $A \frown \text{Left}$ produces 1011". Then you might start suspecting that Left doesn't actually "move the tape head one square to the left", and instead does something like "sum one to the second-to-last non-blank square" (or other plausible hypotheses about what's going on, expressed in the form of a sequence of the real base operations).

I'd like to feed a system data points about the outputs of several different computations (like A , $A \frown \text{Left}$, and so on), and have the system come up with valid hypotheses as to "what the base operations actually do". I'll then be able to use these hypotheses to obtain good guesses about other computations I haven't yet seen: I just run them using my best-guess hypothesis about what the base operations do (or a weighted mixture of them, etc.).

This sounds nice, but a first problem is that I still have to search over the space of hypotheses. I will necessarily have a simplicity bias here, and so I'll end up favoring interpretations of the base operations that are "close to (the simple) reality". This seems like a fundamental conceptual blocker for completely erasing any ground: whatever we do, we'll probably be searching over a space of hypotheses, and at some level (if we have other uncertain meta-levels in between) this search will just have to be implemented by real-world computations, and so will introduce a "real-world simplicity bias".

A second problem (that in my opinion emphasizes the shortcomings of this approach even more), is that by imposing compositionality (and equality), we are already imposing a certain kind of coherence.

Consider the following example. I give you three data points about computations. The first is that A outputs the string 1001. The second is that $A \frown \text{Left}$ outputs 1001. The third is that $A \frown \text{Left} \frown \text{Left}$ outputs 1011. This is a contradiction in our framework: no hypothesis about what Left actually does is consistent with these three data points. If applying Left on 1001 yields 1001, then it cannot be that applying Left on 1001 yields 1011. That's a background assumption we're making about how real computation works.

This might not immediately spell disaster for this idea. The obvious first fix is to allow operations like Left to store information outside the tape (such as a counter, that Left can use to rewrite 1001 only on the second time it's observed). This is a priori unsatisfactory (sounds less naturalistic),

and also makes the simplicity bias kick in even harder, because now our more general hypothesis can overfit more to the observed data points (and so more is decided by the simplicity prior). In any event, let me rush past this interesting layer (and how it might be saved), to continue peeling off.

The above happens because our framework knows that $A \frown \text{Left} \frown \text{Left}$ is the result of concatenating two Lefts after A . But a fundamental (if obvious) realization is that checking for compositionality or equality in this way is already a computation! When we are handed two long strings, and have to check their equality, or compose them and write the result, we are already spending computational resources. I'd like to be uncertain about these propositions as well (for example, about whether $A \frown \text{Left} \frown \text{Left}$ is the result of concatenating two Lefts after A).

Just including those statements as observations (similar to the above "Betting on finite computations") doesn't work, because (again) "the Traders we are using to have opinions over them will still implement real compositionality". That is, saying that "actually the result of concatenating two Lefts after A is $A \frown \text{Right}$ " doesn't make it true inside the Turing Machines we are using. We are not "natively implementing" these "fundamental changes to our logical world". We need something to act "from within".

Weird worlds talk about themselves: The problem seems to be that "the Turing Machines having opinions about the weird worlds are not themselves part of the weird worlds". Let's try to fix that.

Say $\mathcal{S}$ is the space of tape configurations. Different "finite computations" are just functions $f : \mathcal{S} \rightarrow \mathcal{S}$. This is the weird world's equivalent of "run a certain Turing Machine for a certain number of steps".¹ I want to somehow remain agnostic about compositionality. For example, even if I've observed $f(\emptyset) = a$ and $g(\emptyset) = a$, I don't want to impose $h(f(\emptyset)) = h(g(\emptyset))$, or even the particular case $Id(f(\emptyset)) = Id(g(\emptyset))$. I don't yet know how to do that nicely: I can just treat all of them as syntactically different objects, or "=" as a relation not necessarily satisfying some basic axioms of equality. But then they start to become too isolated, without any meaningful interrelationships (because we didn't want to impose "the real relationships").

In any event, brushing past those problems, the punchline is that the f themselves are "our Traders". An obvious proposal is the following:

We fix a way to encode statements about the f in $\mathcal{S}$. Then, we "give points" to the f s having correct opinions about the data points we have observed. For example, if we have observed $g(\emptyset) = 110$, then any f satisfying $f(S_{g(\emptyset)}) = S_{110}$, or $f(S_{g(\emptyset)=110}) = S_{true}$, or something similar, "receives points". We use a sum of all the f s weighted by their points to predict future data points.

A central problem here is that $f(S_{g(\emptyset)}) = S_{110}$ is itself a data point, and the only way for us to "check it" is to receive that data point. In LIs, we can just "run the Trader" and see whether it buys this or that share. But here, since f s don't have any description we can use to compute their opinions, we rely completely on the data point generation to confirm these facts as well. We'd like to do some kind of "iterative delegation": f has the highest score so far, and it believes that f' (about which we barely have any data points yet) will score highly, so we start also believing f' . But the problem remains: each of these "relays" are data points we need to have observed. Since the f have no internal structure, we are left with no way to "extrapolate their opinions". We can impose an external procedure to extrapolate, but this seems like reintroducing real computation.

¹I don't even want to be certain about the number of steps and how they relate to each other, by having f be a TM description, and each f_n is the finite computation of running it for n steps, and I impose something like $f_2 = f_1 \circ f_1$. That'd already impose real compositionality.